IGNOU MCSP-232 PROJECT SYNOPSIS

Automated Cloud Deployment Engine

(ACDE): A Containerized Platform as a Service

SAHIL AGGARWAL

ENROLLMENT NO. 2500675011

PROJECT GUIDE

Aishwariya Singh

Senior Software Engineer

Headout, Bangalore

CORE TECHNOLOGY STACK

Ubuntu | Node.js | Docker | Kubernetes | MongoDB | GCP

PROJECT OVERVIEW

Introduction & Problem Statement

Introduction (The Solution)

The **Automated Cloud Deployment Engine (ACDE)** is a custom Platform as a Service (PaaS) designed to eliminate manual inefficiencies by fully automating the software deployment pipeline.

- ✓ Transforms raw **Git Source Code** directly into a live, accessible **Web URL**.
- ✓ Runs natively on **Ubuntu** using isolated container technology.
- ✓ Automates server provisioning, build processes, and routing configuration.

Problem Definition (The Challenge)

The traditional software deployment lifecycle is plagued by manual overhead, creating bottlenecks and increasing the risk of human error.

- × **Manual Server Provisioning:** Wasted time configuring environments and resolving dependency conflicts.
- × **Complex Routing Setup:** Manual management of reverse proxies for individual applications.
- × **Lack of Visibility:** Fragmented logs make debugging deployment failures difficult.

PROJECT GOALS

Objectives of the Project

Secure Orchestration

Develop a secure Node.js engine capable of executing host-level daemon commands programmatically and safely.

Automated Pipelines

Automate the complete lifecycle: cloning source code, building Docker images, and spinning up isolated containers.

Scalable Orchestration

Demonstrate enterprise-grade scalability using Kubernetes to manage container availability and resources.

Dynamic Reverse Proxy

Implement a system that automatically routes external web traffic to newly assigned internal container ports.

Centralized Dashboard

Provide a unified user interface for monitoring real-time build logs, deployment health, and system status.

SYSTEM DESIGN

Technical Architecture & Stack

Core Stack

Host OS
Ubuntu Linux 20.04+

Backend Engine
Node.js (Express)

Containerization
Docker Engine

Orchestration
Kubernetes (K8s)

Database
MongoDB

Reverse Proxy
Nginx (Dynamic)

Deployment Data Flow



MINIMUM HARDWARE SPECIFICATIONS

PROCESSOR
Intel Core i5 / AMD Ryzen 5

MEMORY
16 GB RAM (Req. for K8s/Docker)

STORAGE
50 GB SSD (Min. Free Space)

DFD Level 0 — Context Diagram

System viewed as a single black-box process with external interactions

Process 0.0 | Scope Context

External Entity: Developer

- Sends deployment request
- Provides Git URL and credentials

External System: Git Provider

- Hosts the source repository
- Supplies branch and code contents

External Entity: Developer (Client)

- Receives generated live URL
- Accesses the deployed app

Diagram Scope

Level 0 does not show internal modules like authentication, orchestration, proxy manager, or database collections. It only shows system boundaries and major data exchanges.

Developer

External Entity

Initiates deployment and monitors the resulting application from the dashboard.

Deploy Request | Git URL | Credentials

INPUT FLOW

Request, repository reference, authentication data

0 ACDE System

Automated Cloud Deployment Engine

Input

Receives user request, Git URL, and credentials.

Core Processing

Fetches code, builds the app, deploys containers, and prepares routing.

Output

Returns deployment status, live URL, and build/runtime logs.

OUTPUT FLOW

Live app URL, status updates, deployment logs

Developer (Client)

External Entity

Receives the live application URL and monitors deployment status and build logs.

Public URL | Build Logs | Status

Data Flows

Major exchanges at context level

Developer → ACDE

Deployment request, Git repository URL, credentials

Git Provider → ACDE

Repository source code and branch contents

ACDE → Developer

Live application URL, deployment status, build logs

System Process | External Entity

DFD Level 1 — Main Processes

System decomposition into 4 core processes with data flows

Process 1.0 | Version 1.0.0

External Entity: User

→ Sends Git URL & Credentials

← Receives Live App URL & Logs

Data Store: MongoDB

Stores: Users, Projects, Deployments, Logs

External: Git Provider

</> Source Code Repository

1 Auth & Dashboard

JWT-based authentication

Login/Register	✓
Session Mgmt	✓
Dashboard	✓

2 Project Configuration

Repo validation & setup

Validate Git URL	✓
Save Configuration	✓
Store Tech Stack	✓

3 Orchestration

Docker/K8s management

Build Images	✓
Deploy Containers	✓
Resource Limits	✓

4 Proxy Manager

Dynamic routing

Update Routes	✓
Subdomain Mapping	✓
Port Allocation	✓

Data Flows

Key interactions between components

User → Auth
Login/Register, View Status

Project Config (Process 2) → Orchestration
Validated repo path, tech stack

Orchestration → Proxy
Port mapping, subdomain

Process · Entity · Data Store

DFD Level 2 — Orchestration Engine Detail

Process 3 decomposition: Clone → Build → Deploy → Update

Process 3.0 | Status Active

Git Provider
- Source Code Repository
- Branch & Commit Info

Docker Daemon
- Image Building
- Container Management

Kubernetes API
- Pod Deployment
- Service Management

MongoDB
- Deployment Records
- Build Logs

3.1 Clone Code — Active
From Git Provider

Clone Repository	✓
Validate URL	✓
Store in Temp	✓

3.2 Build Image — Active
Docker Build

Build Dockerfile	✓
Push to Registry	✓
Image Validation	✓

3.3 Deploy Container — Active
Kubernetes Deploy

Apply Resources	✓
Create Service	✓
Health Check	✓

3.4 Update Database — Active
State & Routing

Insert Record	✓
Update Port	✓
Append Logs	✓

Data Flow: Clone → Build → Deploy → Update

Key interactions between sub-processes

3.1 → 3.2
Source tree, Dockerfile context

3.2 → 3.3
Image reference, build artifacts

3.3 → 3.4
Deployment status, port mapping

Process · Active · Pending

DFD Level 3 — Sub-process Details

Detailed decomposition of deployment and database operations

Process: 3.3 & 3.4 | Version: 1.0.0

3.3 Deploy Container (Kubernetes) — Active

1 Apply Resource Limits ✓	CPU: 500m, Memory: 512Mi	
2 Create Deployment & Service ✓	K8s manifest applied	
3 Health Checks ✓	Readiness probe passed	
4 Emit Logs ✓	Events logged to MongoDB	

3.4 Update Database (State & Routing) — Active

1 Insert Deployment Record ✓	ProjectID, ContainerID, Port	
2 Persist Subdomain ✓	Subdomain mapping saved	
3 Append Build Logs ✓	Timestamps added	
4 Expose Endpoints ✓	Dashboard API ready	

Data Flows & Interactions

K8s API ↔ MongoDB ↔ Proxy Manager

Orchestrator → K8s API
Deployment manifests, service specs
POST /api/v1/namespaces

K8s API → Orchestrator
Status, events, pod info
GET /api/v1/pods

Orchestrator → MongoDB
Deployment records, logs
INSERT/UPDATE

Proxy Manager → User
Live URL, routing
Nginx reload

Health Status

Pod Status	Running
Container ID	acde-1234

Last Updated

Deployment	2 min ago
Logs	Just now

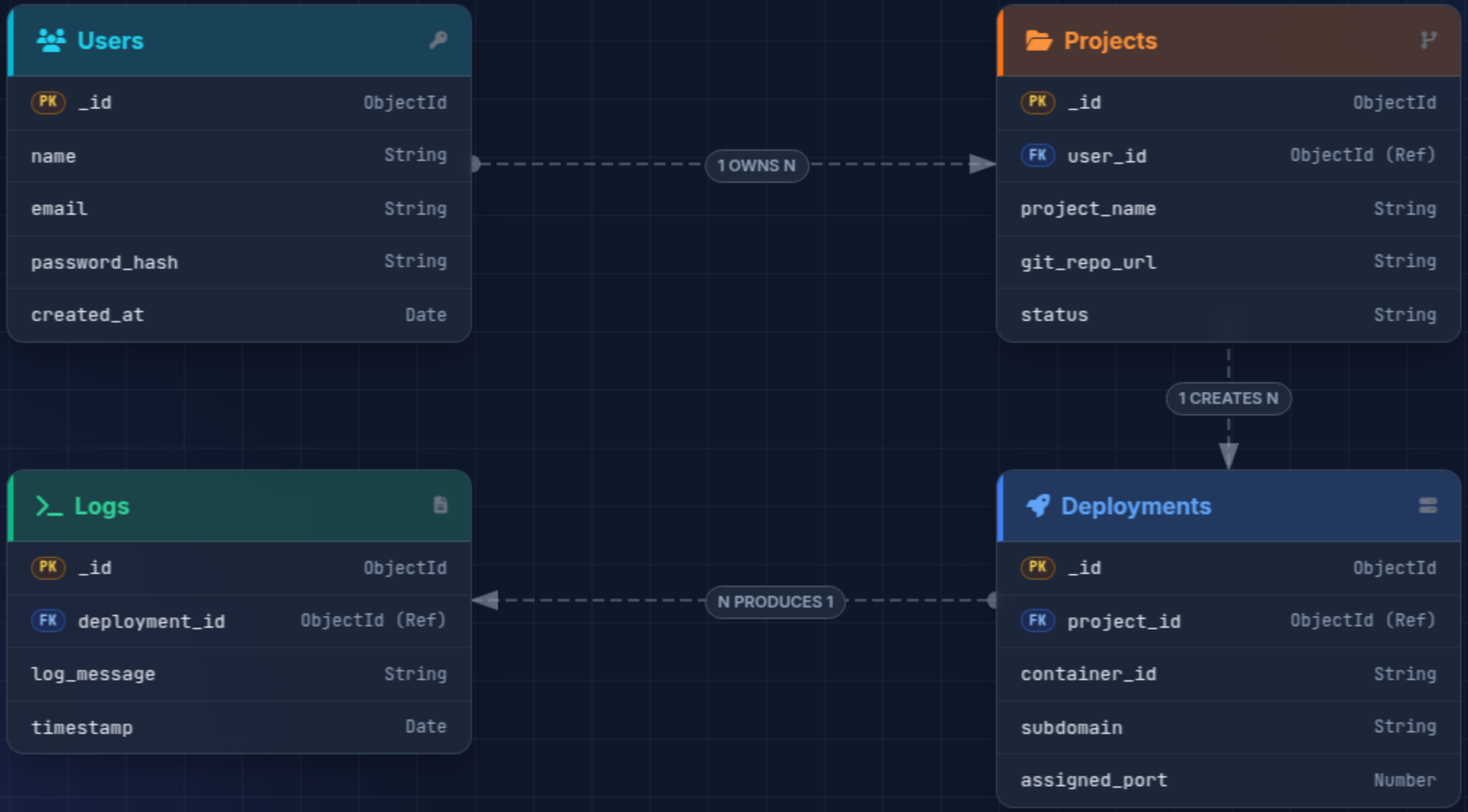
DB Status

Connection	Active
Latency	15ms

DATA MODELS

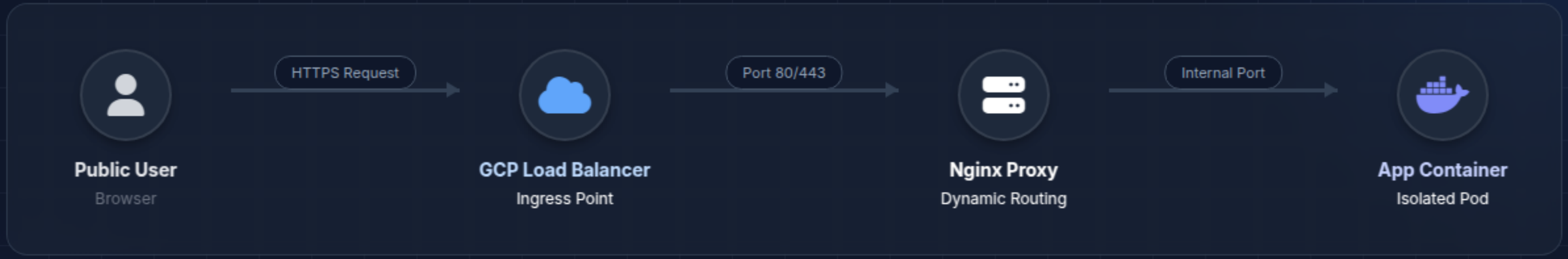
ER Diagram with Cardinality

PK PRIMARY KEY FK FOREIGN KEY



INFRASTRUCTURE & SECURITY

Traffic Request Path



Defense-in-Depth Security Layers

Application Level

- **Bcrypt Hashing:** Secure one-way encryption for all stored user passwords.
- **JWT Authentication:** Stateless token-based access control for API endpoints.
- **Input Validation:** Strict sanitization of Git URLs and project names.

Network Level

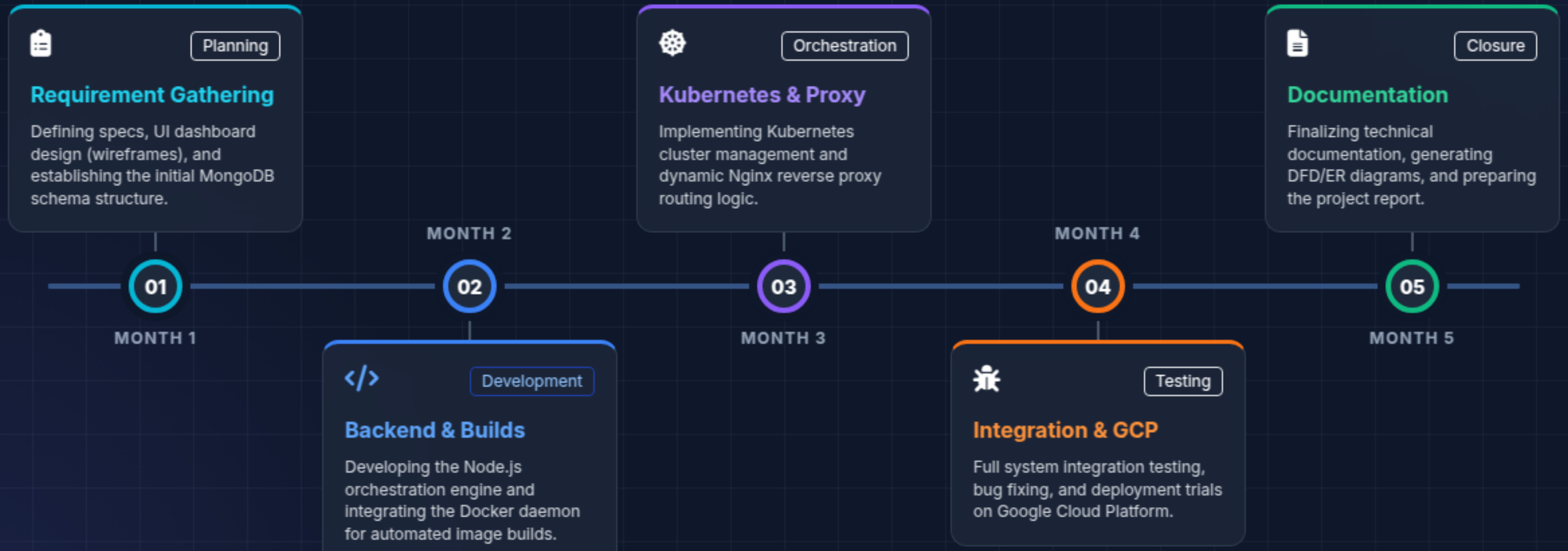
- **Pod Isolation:** Network policies prevent containers from communicating with each other.
- **Internal Routing:** Apps are only accessible via the Nginx reverse proxy, not directly exposed.
- **Firewall Rules:** Strict GCP firewall configuration allowing only essential ports.

System Level

- **Namespace Isolation:** Containers run in isolated Linux namespaces (PID, IPC, Mount).
- **Resource Limits:** Kubernetes enforces CPU and RAM caps to prevent DoS.
- **Restricted FS:** Containers have no access to the Ubuntu host filesystem.

PROJECT SCHEDULE

Implementation Timeline



ROADMAP

Future Scope & Enhancements

Targeting v2.0

Scalability & Automation

While the current iteration establishes the core PaaS foundation, the next phase focuses on seamless enterprise integration, full CI/CD automation, and advanced resource management.



True CI/CD Integration

Implement GitHub/GitLab webhook listeners to automatically trigger new build-and-deploy pipelines whenever code is pushed to the repository branch.

Auto-Provisioned Databases

Enable users to request dedicated database containers (PostgreSQL, MongoDB) attached to their application network with auto-injected environment variables.

Private Repo Authentication

Add OAuth 2.0 and SSH key management to allow cloning from private repositories and organizations, expanding support beyond public open-source projects.

Advanced Monitoring

Integrate Prometheus and Grafana for real-time visualization of container CPU usage, memory leaks, and request latency metrics directly in the user dashboard.

PROCESS MODEL

Software Engineering Paradigm

AGILE / ITERATIVE-INCREMENTAL

SELECTED SE PARADIGM

Agile, built in short validated iterations

ACDE followed an **Agile iterative-incremental model** so build, deployment, and testing could be validated in small increments.

REQUIREMENT STATUS

IGNOU requirement satisfied: the paradigm is stated explicitly as **Agile** and linked to deliverables.

WHY AGILE FITS ACDE

- Requirements, design, build flow, and routing were delivered as short increments.
- Each stage used logs and test results as immediate feedback.

ITERATION MAP

Sprint-Oriented Delivery Sequence

Review -> Build -> Test -> Refine

SPRINT 1

01

Requirements and scope

Defined pain points, actors, and project boundaries.

OUTPUT: PROBLEM STATEMENT

SPRINT 2

02

Architecture and design

Finalized system flow, DFD structure, and collections.

OUTPUT: DESIGN SET

SPRINT 3

03

Core build automation

Implemented repository processing and Docker build logic.

OUTPUT: BUILD FLOW

SPRINT 4

04

Deployment integration

Connected Kubernetes deployment, routing, and logging.

OUTPUT: LIVE PIPELINE

SPRINT 5

05

Testing and closure

Executed verification and closed the report set.

OUTPUT: FINAL SUBMISSION

AGILE OUTCOME ACDE moved from concept to tested deployment through short feedback-driven iterations instead of a rigid Waterfall handoff.

PROJECT SCHEDULING

Gantt Chart and PERT Chart

MANDATORY PLANNING ARTEFACTS

GANTT VIEW

Five-Month Execution Plan

Timeline aligned with the existing implementation schedule already present in the deck.

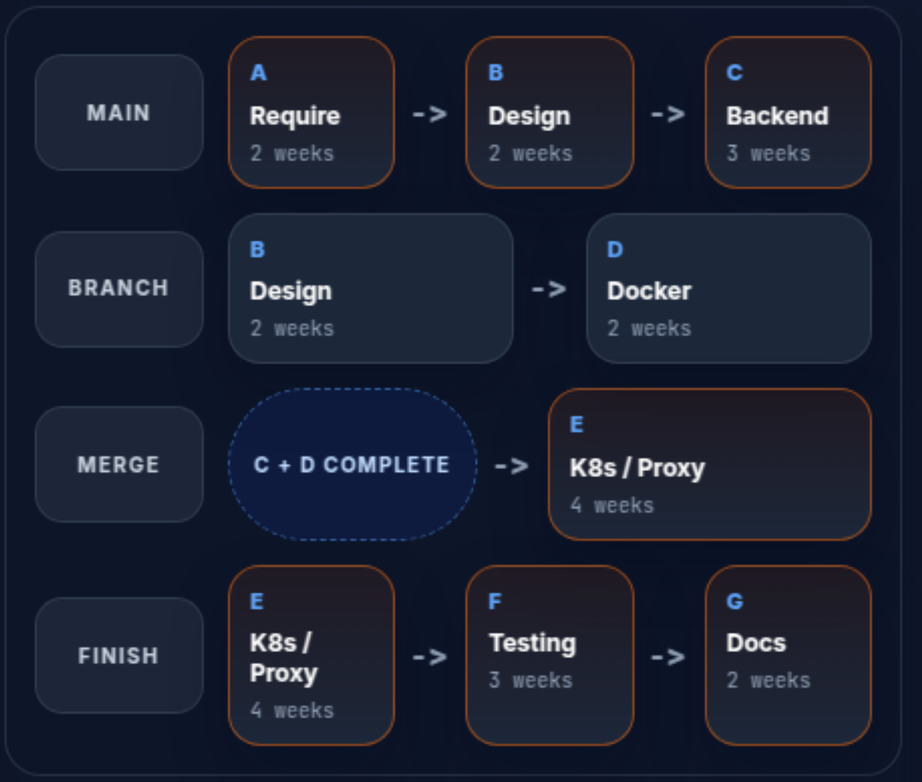
TASK	MONTH 1	MONTH 2	MONTH 3	MONTH 4	MONTH 5
Requirement analysis	Plan				
System design and DB schema	Design				
Backend and API development		Develop			
Docker build automation		Build			
Kubernetes and Nginx integration			Deploy		
Security hardening and validation			Harden	Verify	
Integration testing and GCP trial				Test	
Documentation and final report					Close

- GANTT MAPS TASKS TO TIME
- PERT SHOWS DEPENDENCIES
- TOTAL PLANNED DURATION: 5 MONTHS

PERT VIEW

Task Dependency Network

CRITICAL PATH = 16 WEEKS



CRITICAL PATH A → B → C → E → F → G. Task D branches from B and must finish before E starts, but it is not on the longest path.

QUALITY ASSURANCE

Testing Plan and Sample Test Cases

MANDATORY TEST PLAN INCLUDED

APPROACH

Multi-Level Verification

ACDE is verified through validation checks, integration flow tests, and deployment security review before submission.

UNIT AND INPUT CHECKS

Validate login handling, Git URL format, and request sanitization.

INTEGRATION FLOW

Confirm cloning, image creation, status updates, and MongoDB writes.

SYSTEM AND SECURITY

Verify live routing, failure logging, and ownership controls.

PASS

Expected status and records are produced.

FAIL

Errors are logged and status is marked failed.

EXIT

All cases pass before final submission.

SAMPLE CASES

Representative Test Case Matrix

Five concise cases cover success flow, rejection handling, failure logging, and access control.

ID	LEVEL	SCENARIO	TEST INPUT / STEPS	EXPECTED RESULT
TC-01	UNIT	Valid login	Submit correct email and password to the login API.	JWT is returned and the user session is accepted.
TC-02	UNIT	Invalid repository input	Send a malformed Git URL in the deployment request payload.	Validation fails and no deployment record is created.
TC-03	INTEGRATION	Successful build pipeline	Provide a valid repository and trigger deployment from the dashboard.	Repository is cloned, image is built, and status moves to success.
TC-04	SYSTEM	Failed deployment logging	Deploy a repository with a forced dependency error.	Failure is recorded in Logs and the deployment is marked failed.
TC-05	SECURITY	Unauthorized project access	Log in as another user and request a project you do not own.	Access is denied with an authorization error and no data is exposed.

Thank You

Automated Cloud Deployment Engine (ACDE)

Secure Automation

Automated pipelines secured with JWT authentication and bcrypt hashing.

Container Isolation

Namespace-isolated execution environments for complete safety.

K8s Scalability

Enterprise-grade orchestration ready for Google Cloud Platform.

Dynamic Routing

Real-time subdomain mapping via Nginx reverse proxy.

Questions?